

VayuSangam — Technical Demo Specification

SIH2026 • SIH26082 • Air Pollution–Weather Coupled Forecasting System (Delhi NCR Focus)

VayuSangam --- Technical Demo Design

1. Demo objective

Build a credible, runnable prototype for SIH26082 --- Air Pollution--Weather Coupled Forecasting System (Delhi NCR Focus).

The demo must visually and technically prove this chain:

Weather + satellite + fire + ground data → source intelligence → coupled/physics-informed forecast representation → fast ML surrogate → local correction → 72 h PM_{2.5}/O₃/AQI forecast → inversion/plume/source explanation → what-if intervention.

The prototype must work in two modes:

1. Demo Mode (mandatory): uses bundled/sample NetCDF/CSV/GeoJSON data and pre-generated WRF-Chem-like forecast tensors so the UI works without HPC or API credentials.
2. Live/Research Mode (optional): adapters for FIRMS, Sentinel-5P/Copernicus, CPCB/ground observations and actual WRF-Chem output can be enabled later.

Do not fake a completed WRF-Chem run. The UI must label demo/synthetic data clearly.

2. Repository strategy

Use official upstream repositories as the scientific foundation.

Clone these

```
git clone --recurse-submodules https://github.com/wrf-model/WRF.git third_party/WRF
```

```
git clone https://github.com/wrf-model/WPS.git third_party/WPS
```

```
git clone https://github.com/noaa-oar-arl/utilhysplit.git third_party/utilhysplit
```

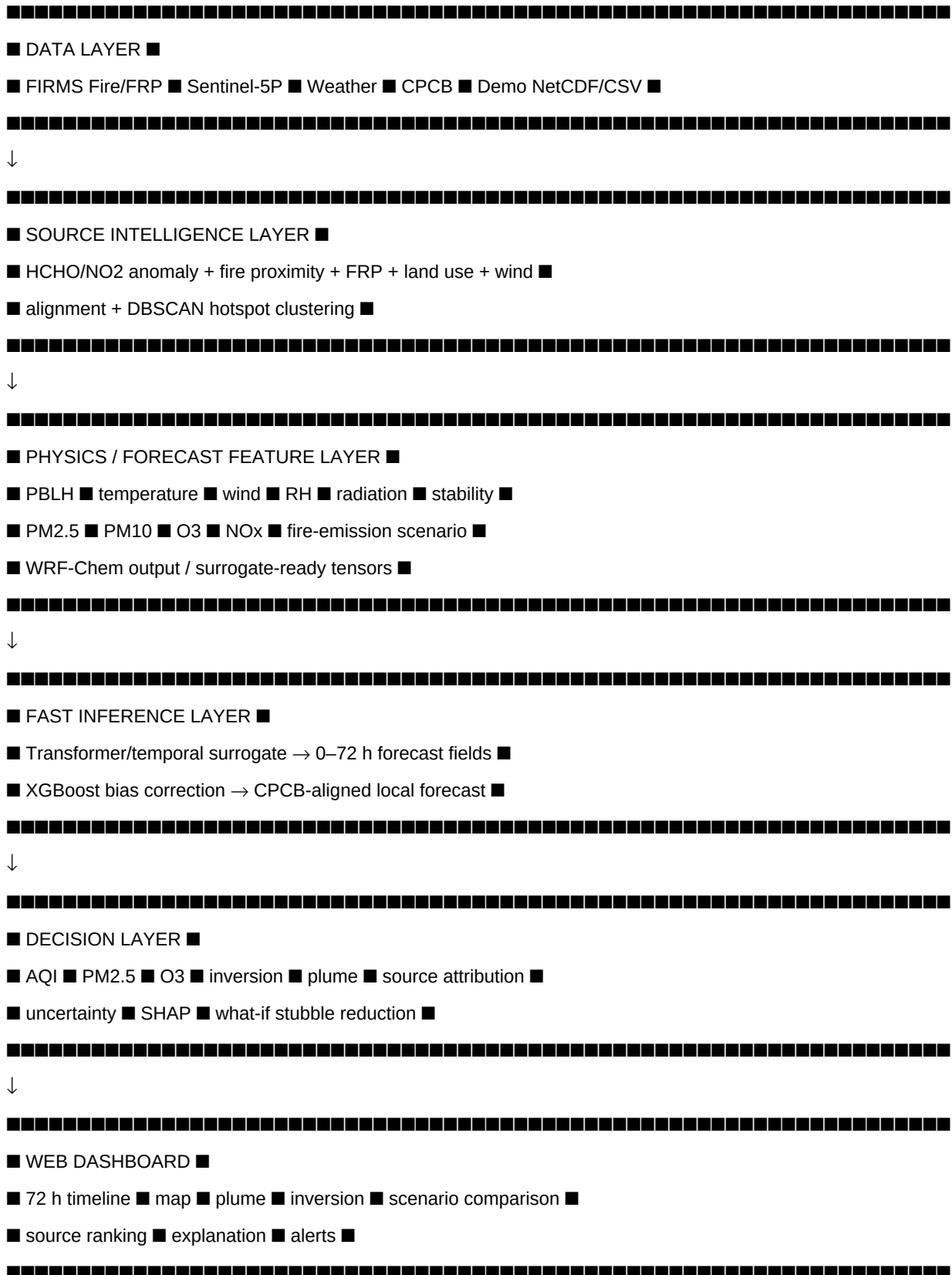
WRF contains the WRF-Chem chemistry code. Do not clone an arbitrary GitHub fork named `WRF-Chem`; use the official `wrf-model/WRF` repository.

WPS prepares real-data WRF inputs. `utilhysplit` is useful for HYSPLIT input/output processing and plume/trajectory utilities.

The first demo should NOT require compiling WRF on the developer laptop.

Keep WRF/WPS isolated under `third\_party/` and provide an optional Linux/WSL/HPC build path.

3. High-level architecture



4. Physics/ML boundary

WRF-Chem = physics/ground-truth engine

Use WRF-Chem for offline/batch research runs:

- meteorology
- transport
- gas/aerosol chemistry
- emissions
- aerosol-radiation feedback
- PM2.5/PM10/O3/NOx fields

Its outputs become training/validation examples for the fast inference layer.

Transformer surrogate = real-time engine

Train a temporal/spatiotemporal model on WRF-Chem-like tensors.

Minimum MVP:

- input: 6--12 historical/initial feature frames
- horizon: 72 hourly steps
- output: PM2.5, O3, PBLH, plume probability/transport field
- architecture: small Conv/Temporal Transformer or ConvLSTM-style model
- inference: CPU-compatible first; GPU optional

For a hackathon demo, the surrogate may initially be a trained/simple model over prepared WRF-Chem-derived samples. It must have the same API contract as a future production transformer.

XGBoost = local correction

Train an XGBoost regressor using:

- surrogate PM2.5/O3
- CPCB observations
- temperature
- wind speed/direction
- RH
- PBLH
- hour/day/season
- source/fire indicators

Output:

`corrected\_pm25`, `corrected\_o3`, confidence/uncertainty estimate.

5. Source intelligence

For each detected fire/source candidate calculate:

- latitude/longitude
- FRP
- fire confidence
- distance to Delhi NCR
- wind direction
- wind alignment score
- estimated travel time
- HCHO anomaly score
- NO2 anomaly score where available
- agricultural-land proximity
- cluster ID
- source contribution score

A source should only be called a likely contributor when multiple signals agree.

Example score:

```
source_score =  
0.30 * normalized_FRP  
+ 0.20 * HCHO_anomaly  
+ 0.15 * NO2_anomaly  
+ 0.15 * wind_alignment  
+ 0.10 * proximity  
+ 0.10 * agricultural_land_score
```

Keep weights configurable in YAML. Do not present this heuristic as scientifically calibrated unless validated.

6. Inversion intelligence

Calculate a demo inversion/trapping index from:

- PBL height
- near-surface temperature

- vertical temperature gradient when available
- wind speed
- stability proxy

Return:

- `Weak`
- `Moderate`
- `Strong`
- `Severe`

Also return the numeric index and contributing variables.

The UI should explain:

Stronger inversion + shallow PBL + weak winds → reduced dispersion →
higher accumulation risk.

7. What-if engine

The most important live demo interaction:

Scenario A: Stubble emissions = 100%

Scenario B: Stubble emissions = 70%

Scenario C: Stubble emissions = 40%

The user moves a slider.

The system modifies the source/emission feature tensor and runs the surrogate again.

Display:

- Delhi NCR PM2.5
- AQI
- O3
- peak hour
- plume intensity
- source contribution
- change vs baseline

Do not rerun WRF-Chem on every slider move.

8. API design

``GET /api/health``

Returns service status.

`GET /api/forecast?hours=72`

Returns forecast time series and metadata.

`GET /api/forecast/grid?hour=24&variable;=pm25`

Returns spatial grid for map rendering.

`GET /api/sources?hour=24`

Returns source clusters and attribution scores.

`GET /api/inversion?hour=24`

Returns inversion/trapping metrics.

`POST /api/scenario`

Request:

```
{
  "stubble_reduction": 0.30,
  "hour": 24
}
```

Response:

```
{
  "baseline_pm25": 178.4,
  "scenario_pm25": 141.7,
  "pm25_change": -36.7,
  "baseline_aqi": 312,
  "scenario_aqi": 248,
  "peak_hour_shift": 2
}
```

`GET /api/explanation?hour=24`

Returns SHAP/top-feature explanations.

9. Dashboard layout

Top bar

- VayuSangam
- Delhi NCR

- Demo/Live badge
- data timestamp

KPI row

- Current PM2.5
- 24 h peak
- 72 h peak
- O3
- Inversion strength

Main map

Layers:

1. PM2.5 heatmap
2. fire/FRP points
3. HCHO hotspot clusters
4. wind vectors
5. plume trajectory
6. Delhi NCR boundary

Timeline

0--72 h slider.

Right panel

- Source attribution
- inversion explanation
- uncertainty
- SHAP features

What-if panel

Slider:

`Stubble emissions: 100% → 0%`

Compare baseline vs intervention.

10. Technology stack

Backend

- Python 3.11+
- FastAPI
- Pydantic

- NumPy
- pandas
- xarray
- netCDF4/h5netcdf
- scikit-learn
- XGBoost
- PyTorch
- GeoPandas
- Shapely

Frontend

- Next.js
- TypeScript
- Tailwind CSS
- MapLibre GL JS
- Recharts
- Zustand or React state

Data formats

- NetCDF/Zarr for gridded fields
- Parquet for tabular observations
- GeoJSON for boundaries/hotspots
- JSON for API responses

11. Directory structure

vayu-sangam/

■■■■ backend/

■ ■■■■ app/

■ ■ ■■■■ main.py

■ ■ ■■■■ api/

■ ■ ■■■■ models/

■ ■ ■■■■ services/

■ ■ ■■■■ schemas/

■ ■■■■ data/

■ ■ ■■■■ demo/

■ ■ ■■■■ processed/

■ ■■■■ ml/

■ ■ ■■■■ surrogate/

■ ■ ■■■■ correction/

■ ■ ■■■■ explainability/

- ■■■ pipelines/
- ■■■ tests/
- frontend/
- ■■■ app/
- ■■■ components/
- ■■■ lib/
- ■■■ public/
- configs/
- ■■■ demo.yaml
- ■■■ live.yaml
- scripts/
- notebooks/
- third_party/
- ■■■ WRF/
- ■■■ WPS/
- ■■■ utilhysplit/
- docs/
- requirements.md
- design.md
- task.md
- IDE_PROMPT.md

12. Demo truthfulness rules

- Never call synthetic values "real-time satellite data".
- Clearly mark sample/demo data.
- Never claim an ML metric until it has been computed on a held-out dataset.
- Never claim WRF-Chem was run if only surrogate/demo tensors are being used.
- Keep data provenance visible in the UI.
- Make live adapters optional so the demo remains reproducible.

VayuSangam --- Requirements

1. Product requirement

Build a working technical prototype for SIH26082 that demonstrates 72-hour Delhi NCR air-pollution/weather coupled forecasting with source attribution, inversion intelligence, and intervention simulation.

The prototype must prioritize a working end-to-end demo over a large unfinished research stack.

2. Functional requirements

FR-01 --- 72-hour forecast

The system shall display hourly 0--72 h:

- PM2.5
- O3
- PM10
- NOx where available
- temperature
- wind speed/direction
- PBL height
- inversion/trapping level

FR-02 --- Spatial forecast

The system shall render an hourly spatial field for at least:

- PM2.5
- O3

The MVP grid can be demo data, but the data contract must support NetCDF/Zarr-derived grids.

FR-03 --- Fire intelligence

The system shall ingest or simulate:

- fire latitude/longitude
- FRP
- confidence
- timestamp

The UI shall render fire points and intensity.

FR-04 --- HCHO/source intelligence

The system shall support HCHO anomaly fields or demo HCHO hotspot data and combine them with:

- FRP
- wind
- distance
- agricultural-land indicator

to produce source clusters and a ranked source-attribution list.

FR-05 --- Plume tracking

The system shall display a source-to-Delhi transport path using wind/trajectory data.

For MVP, a deterministic trajectory approximation is acceptable, provided it is clearly labeled as a prototype transport layer.

FR-06 --- Inversion intelligence

The system shall calculate and display:

- inversion/trapping score
- PBL height
- wind speed
- stability category

Categories:

`Weak / Moderate / Strong / Severe`

FR-07 --- Fast forecast

The UI shall receive forecast results from a fast inference service.

The API must be model-agnostic so a Transformer surrogate can replace the MVP model without changing the frontend.

FR-08 --- Bias correction

An XGBoost correction layer shall be supported.

Inputs should include:

- raw/surrogate PM2.5
- raw/surrogate O3
- meteorological variables

- PBLH
- source indicators
- temporal features

FR-09 --- What-if scenario

The user shall be able to change stubble-burning intensity.

At minimum:

- 100%
- 70%
- 40%

The system shall immediately recompute forecast outputs through the surrogate/scenario layer.

FR-10 --- Comparison

The UI shall show:

- baseline PM2.5
- scenario PM2.5
- delta
- baseline AQI
- scenario AQI
- delta percentage

FR-11 --- Explainability

The UI shall show the top forecast drivers, e.g.:

- PBL height
- wind speed
- fire/FRP
- HCHO anomaly
- temperature
- humidity

If SHAP is available, use SHAP. Otherwise use a clearly labeled feature-importance fallback.

FR-12 --- Data provenance

Every forecast screen shall expose:

- Demo / Live status
- source
- timestamp

- model name/version
 - whether output is surrogate or WRF-Chem
-

3. Non-functional requirements

NFR-01 --- Reproducibility

A fresh developer must be able to start Demo Mode using documented commands.

NFR-02 --- Offline-first demo

The dashboard must work without internet after dependencies and demo assets are installed.

NFR-03 --- Performance

Target:

- API health ≤ 500 ms
- normal forecast query ≤ 2 s
- what-if inference ≤ 5 s on a normal development machine

These are engineering targets, not scientific accuracy claims.

NFR-04 --- Responsive UI

Dashboard must work at:

- 1366×768
- 1920×1080

NFR-05 --- Error handling

No blank screens.

Show actionable messages for:

- missing data
- model unavailable
- API failure
- invalid scenario
- missing environment variable

NFR-06 --- Model abstraction

Model interface:

class ForecastModel:

```
def predict(self, features, horizon=72):
```

```
...
```

Implement:

- `DemoForecastModel`
- `SurrogateForecastModel`

so the frontend/API never depends directly on PyTorch internals.

4. Scientific requirements

The prototype should represent the coupled feedback concept:

Weather

↓

Transport / PBL / radiation

↓

Pollutant concentration

↓

Aerosol loading

↓

Radiation modification

↓

Temperature / PBL change

↓

Transport + chemistry change

The MVP may represent this relationship through WRF-Chem-derived training features and a surrogate model. Do not claim the browser is performing a full two-way WRF-Chem integration.

5. Data requirements

Demo data

Create a small reproducible dataset covering Delhi NCR and nearby source regions.

Minimum files:

forecast_demo.nc

fires_demo.csv

hcho_hotspots.geojson

cpcb_demo.csv
wind_demo.nc
delhi_ncr.geojson
source_clusters.geojson

Live adapters

Optional:

- NASA FIRMS API
- Sentinel-5P/Copernicus data
- CPCB observations
- ERA5/NWP input
- WRF-Chem NetCDF output

API credentials must be environment variables, never committed.

6. Acceptance criteria

The demo is accepted only when a judge can perform this sequence:

1. Open dashboard.
 2. See Delhi NCR forecast.
 3. Move 0--72 h slider.
 4. See PM2.5 map change.
 5. Turn on fire layer.
 6. Select a source cluster.
 7. See its FRP/HCHO/wind/source score.
 8. See plume direction toward Delhi.
 9. See inversion strength change over time.
 10. Move stubble slider from 100% to 40%.
 11. See PM2.5/AQI update within 5 seconds.
 12. See baseline-vs-scenario comparison.
 13. Open explanation and see why the peak occurred.
 14. Confirm whether the screen is Demo or Live.
-

7. Security

- `.env` is gitignored.
- API keys never enter frontend source.
- No secrets in notebooks.
- CORS restricted in production.

- Uploaded files validated by type and size.

8. Out of scope for first demo

Do NOT block the demo on:

- full operational WRF-Chem compilation
- real-time ingestion of every satellite product
- nationwide 1 km forecasting
- perfect AQI regulatory certification
- mobile application
- multi-cloud deployment
- full chemistry data assimilation

These belong to later phases.

VayuSangam --- Implementation Task Plan

Phase 0 --- Repository setup

T0.1 Create project

- [] Create `vayu-sangam/`
- [] Initialize Git
- [] Add `.gitignore`
- [] Add Python virtual environment instructions
- [] Add Node/Next.js setup

T0.2 Clone scientific dependencies

```
git clone --recurse-submodules https://github.com/wrf-model/WRF.git third_party/WRF
git clone https://github.com/wrf-model/WPS.git third_party/WPS
git clone https://github.com/noaa-oar-arl/utilhysplit.git third_party/utilhysplit
```

- [] Verify repositories
 - [] Do not modify upstream source initially
 - [] Document Linux/WSL/HPC compilation separately
-

Phase 1 --- Demo data

T1.1 Create reproducible Delhi grid

- [] Define Delhi NCR bounding box
- [] Generate 0.05°--0.1° demo grid
- [] Create 73 hourly timestamps
- [] Store latitude/longitude arrays

T1.2 Generate weather features

Create realistic-looking but explicitly synthetic demo fields:

- [] temperature
- [] wind u/v
- [] RH
- [] PBLH
- [] radiation/stability proxy

Output:

`backend/data/demo/wind\_demo.nc`

T1.3 Generate pollution fields

- [] PM2.5
- [] O3
- [] PM10
- [] NOx

Use spatial Gaussian source plumes plus temporal evolution.

Output:

`backend/data/demo/forecast\_demo.nc`

T1.4 Generate fire data

Create:

- [] 20--100 fire points
- [] FRP
- [] confidence
- [] timestamps

Output:

`fires\_demo.csv`

T1.5 Generate HCHO hotspots

- [] Create hotspot polygons/points
- [] HCHO anomaly score
- [] cluster ID

Output:

`hcho\_hotspots.geojson`

Phase 2 --- Source intelligence

T2.1 Fire clustering

- [] DBSCAN on fire coordinates
- [] cluster FRP sum/mean/max
- [] cluster centroid

T2.2 Wind alignment

- [] Calculate bearing from source to Delhi
- [] Calculate wind-direction difference
- [] Convert to alignment score

T2.3 Travel-time estimate

- [] Estimate source-to-Delhi distance
- [] Estimate travel time from wind speed
- [] Return uncertainty band

T2.4 Source score

Implement configurable score from `configs/demo.yaml`.

- [] FRP
- [] HCHO
- [] NO2 if available
- [] wind alignment
- [] proximity
- [] agriculture score

Phase 3 --- Inversion intelligence

- [] Implement PBLH normalization
- [] Implement wind-speed normalization
- [] Implement stability proxy
- [] Create inversion index
- [] Map to four categories
- [] Unit-test thresholds

Phase 4 --- Forecast engine

T4.1 Model interface

Create:

```
class ForecastModel:
    def predict(self, features, horizon=72):
        raise NotImplementedError
```

T4.2 Demo model

- ☐ Load demo NetCDF
- ☐ Return forecast tensor
- ☐ Add deterministic scenario multiplier

T4.3 Surrogate model

- ☐ Build training dataset interface
- ☐ Implement PyTorch temporal model
- ☐ Save model checkpoint
- ☐ Implement inference
- ☐ Add CPU fallback

T4.4 XGBoost correction

- ☐ Build feature matrix
- ☐ Train baseline XGBoost
- ☐ Save model
- ☐ Implement prediction
- ☐ Add feature importance

Do not report performance metrics unless a proper train/validation/test split has been performed.

Phase 5 --- Scenario engine

Implement:

baseline = 1.0

moderate = 0.70

aggressive = 0.40

- ☐ Modify emission features
- ☐ Run surrogate
- ☐ Apply XGBoost correction
- ☐ Recalculate AQI
- ☐ Return delta
- ☐ Cache repeated scenarios

Phase 6 --- Backend

T6.1 FastAPI

- [] `/api/health`
- [] `/api/forecast`
- [] `/api/forecast/grid`
- [] `/api/sources`
- [] `/api/inversion`
- [] `/api/scenario`
- [] `/api/explanation`

T6.2 Validation

- [] Pydantic schemas
- [] invalid hour handling
- [] variable whitelist
- [] scenario range 0--1

T6.3 Caching

- [] Cache demo forecast
- [] Cache grid tiles/JSON
- [] Cache scenario responses

Phase 7 --- Frontend

T7.1 Dashboard shell

- [] Next.js
- [] responsive layout
- [] header
- [] KPI cards

T7.2 Map

- [] Delhi NCR base map
- [] PM2.5 raster/heatmap
- [] fire points
- [] HCHO clusters
- [] wind vectors
- [] plume line

T7.3 Timeline

- ☐ 0--72 h slider
- ☐ play/pause
- ☐ selected hour

T7.4 Intelligence panels

- ☐ inversion card
- ☐ source attribution
- ☐ explanation
- ☐ uncertainty

T7.5 What-if

- ☐ stubble slider
- ☐ baseline card
- ☐ scenario card
- ☐ delta visualization

Phase 8 --- Demo polish

- ☐ Add Demo/Live badge
- ☐ Add data provenance
- ☐ Add model version
- ☐ Add loading states
- ☐ Add error states
- ☐ Add empty states
- ☐ Add "last updated"
- ☐ Add unit labels
- ☐ Remove scientific claims that are not validated

Phase 9 --- Judge flow

Prepare a 3-minute deterministic demo:

0:00--0:30

Show current Delhi NCR PM2.5/O3 and 72 h outlook.

0:30--1:00

Move time slider to the predicted peak.

Show:

- shallow PBL
- weak wind
- inversion
- PM2.5 increase

1:00--1:30

Enable source layer.

Click a Punjab/Haryana-region source cluster.

Show:

- FRP
- HCHO anomaly
- wind alignment
- travel-time estimate
- plume toward Delhi

1:30--2:15

Open what-if.

Change stubble emissions:

`100% → 70% → 40%`

Show PM2.5/AQI reduction.

2:15--2:45

Open explanation.

Show top drivers and SHAP/feature importance.

2:45--3:00

Show architecture/data provenance and explain:

WRF-Chem = physics/ground truth Surrogate = fast inference

XGBoost = local correction

Phase 10 --- Final verification

- [] `docker compose up` works if Docker is provided
- [] `npm run build` passes
- [] backend tests pass

- ☐ frontend loads with backend unavailable
- ☐ demo mode works offline
- ☐ no secret committed
- ☐ no overlapping dashboard elements
- ☐ no fake "real-time" claims
- ☐ all API endpoints documented

IDE MASTER PROMPT --- VayuSangam Technical Demo

You are the lead full-stack + atmospheric-science engineer implementing VayuSangam, a technical prototype for SIH2026 Problem Statement SIH26082: Air Pollution--Weather Coupled Forecasting System (Delhi NCR Focus).

Read these files first and treat them as the project specification:

1. `design.md`
2. `requirements.md`
3. `task.md`

Do not skip them.

Primary goal

Build a working, judge-ready technical demo, not a collection of disconnected mock screens.

The judge must be able to:

1. view a 72-hour Delhi NCR PM2.5/O3 forecast,
2. move through time,
3. see fire/HCHO source intelligence,
4. see wind/plume transport,
5. see inversion/PBL conditions,
6. understand why pollution peaks,
7. change stubble-burning intensity,
8. see the forecast update quickly,
9. compare baseline vs intervention,
10. understand the model/data provenance.

Scientific architecture

Use this conceptual pipeline:

`Weather + Satellite + Fire + Ground → Source Intelligence → Physics/WRF-Chem Feature Space → Fast Surrogate → XGBoost Correction → Forecast + Attribution + What-if`

Important:

- WRF-Chem is the physics/ground-truth engine.
- The browser dashboard must NOT run WRF-Chem interactively.

- A Transformer/temporal surrogate is the fast inference engine.
- XGBoost is the local observation-bias correction layer.
- Demo Mode must work without WRF-Chem compilation.
- Never claim synthetic data is real-time.
- Never claim scientific accuracy without validation.

Scientific repositories

Use official repositories:

```
git clone --recurse-submodules https://github.com/wrf-model/WRF.git third_party/WRF
```

```
git clone https://github.com/wrf-model/WPS.git third_party/WPS
```

```
git clone https://github.com/noaa-oar-arl/utilhysplit.git third_party/utilhysplit
```

Do NOT replace the official WRF repository with an arbitrary GitHub fork.

WRF contains WRF-Chem. WPS is the official preprocessing system.

`utilhysplit` can support HYSPLIT trajectory/input-output processing.

Do not block the first runnable demo on compiling WRF.

Implementation rules

Rule 1 --- Build in vertical slices

Implement one complete flow first:

```
`demo data → FastAPI → dashboard map → timeline → scenario`
```

Only after that add deeper ML/science components.

Rule 2 --- Real APIs are adapters

Create interfaces:

```
class FireDataProvider:
```

```
def get_fires(self, start, end, bbox): ...
```

```
class SatelliteDataProvider:
```

```
def get_hcho(self, start, end, bbox): ...
```

```
class GroundDataProvider:
```

```
def get_observations(self, start, end, stations): ...
```

Implement:

- `DemoFireProvider`
- `DemoSatelliteProvider`
- `DemoGroundProvider`

Later implement live providers.

Rule 3 --- Model abstraction

Create:

```
class ForecastModel:
def predict(self, features, horizon=72):
...
```

Implement:

- `DemoForecastModel`
- `SurrogateForecastModel`

The API must not know PyTorch internals.

Rule 4 --- Scenario abstraction

```
class ScenarioEngine:
def run(self, stubble_fraction: float, hour: int):
...
```

Validate `0 <= stubble\_fraction <= 1`.

Rule 5 --- Data provenance

Every response should contain:

```
{
"mode": "demo",
"data_source": "bundled_demo_dataset",
"model": "DemoForecastModel",
"model_version": "0.1.0",
"timestamp": "...",
"scientific_status": "prototype"
}
```

Frontend expectations

Make the dashboard look like a serious atmospheric-intelligence platform.

Avoid:

- generic admin-dashboard appearance,
- excessive gradients,
- random bright colors,
- giant cards,

- clutter,
- tiny labels,
- fake scientific charts with no units.

Use:

- dark/neutral map,
- restrained blue/cyan/orange warning palette,
- clear typography,
- strong spatial hierarchy,
- compact KPI cards,
- map-first layout.

Main screen

Top:

- VayuSangam
- Delhi NCR
- Demo/Live
- model version
- timestamp

KPI row:

- PM2.5
- AQI
- O3
- PBLH
- Inversion

Center:

- large map
- PM2.5 layer
- fire points
- HCHO clusters
- wind vectors
- plume

Bottom:

- 0--72 h timeline

Right:

- source attribution
- inversion explanation
- top drivers

- uncertainty

Bottom/right:

- What-if stubble emission slider

Required API endpoints

Implement:

- `GET /api/health`
- `GET /api/forecast?hours=72`
- `GET /api/forecast/grid?hour=24&variable;=pm25`
- `GET /api/sources?hour=24`
- `GET /api/inversion?hour=24`
- `POST /api/scenario`
- `GET /api/explanation?hour=24`

Use typed Pydantic schemas.

Demo dataset

Generate a reproducible Delhi NCR dataset.

Do not use arbitrary random noise.

Create spatially coherent synthetic fields:

- PM2.5 plume
- O3 field
- wind field
- PBLH cycle
- fire clusters
- HCHO hotspots
- source-to-Delhi transport

The demo should have at least one visually obvious pollution episode.

For example:

- source region north-west of Delhi,
- stronger fire activity before the episode,
- wind transporting the plume toward Delhi,
- shallow PBL/strong inversion around the PM2.5 peak.

Label this as `DEMO / SYNTHETIC`.

What-if behavior

When user changes:

`100% → 70% → 40%`

the system should:

1. modify source emission features,
2. run the fast model,
3. apply correction,
4. recompute PM2.5/AQI,
5. update map/timeline/KPI,
6. show delta vs baseline.

Target inference time: ≤ 5 seconds.

Do not rerun WRF-Chem.

AQI

Implement an explicit configurable AQI conversion module.

Do not bury AQI calculations inside frontend code.

Document the pollutant breakpoints/source used for the prototype.

If regulatory breakpoint data is not included in the repository, use a clearly labeled prototype configuration and do not claim regulatory certification.

ML

First make the system work with `DemoForecastModel`.

Then implement:

Surrogate

A compact temporal model using PyTorch.

Keep it small enough to run locally.

Inputs:

- PM2.5
- O3
- temperature
- wind u/v
- RH
- PBLH
- fire/FRP

- HCHO anomaly
- hour features

Outputs:

- PM2.5
- O3
- PBLH/plume features where useful

XGBoost

Use it as local correction.

Persist model artifacts under:

``backend/ml/artifacts/``

Do not commit huge model binaries unless necessary.

Testing

Write tests for:

- source score
- wind alignment
- inversion classification
- AQI conversion
- scenario bounds
- forecast endpoint
- scenario endpoint
- model interface

Add one end-to-end test:

``scenario request → forecast response → changed PM2.5``

Development sequence

Execute in this order:

Step 1

Create repository structure.

Step 2

Create demo dataset.

Step 3

Implement source intelligence.

Step 4

Implement inversion engine.

Step 5

Implement DemoForecastModel.

Step 6

Implement FastAPI endpoints.

Step 7

Implement dashboard.

Step 8

Implement scenario slider.

Step 9

Add explanation/feature importance.

Step 10

Add optional Transformer surrogate.

Step 11

Add optional live data adapters.

Step 12

Write complete setup documentation.

Do not jump to Step 10 before Steps 1--8 work.

Definition of done

Run:

backend

uvicorn backend.app.main:app --reload

frontend

npm run dev

Then verify:

- dashboard opens,

- 72h slider works,
- map changes with time,
- source clusters appear,
- plume appears,
- inversion changes,
- scenario slider works,
- PM2.5 changes,
- baseline/scenario comparison works,
- explanation works,
- Demo badge is visible,
- no API key is required for Demo Mode.

Final instruction

When you finish each phase, report:

1. files created,
2. files changed,
3. commands executed,
4. tests passed,
5. known limitations,
6. exact next step.

Never silently replace scientific components with fake claims.

If a component cannot be implemented scientifically in the local environment, implement a clean interface + reproducible demo implementation and clearly mark it as prototype.

README_DEMO.md

VayuSangam Technical Demo Starter

This package contains the engineering specification for the VayuSangam SIH26082 technical demo.

Scientific repositories

- WRF: <https://github.com/wrf-model/WRF>
- WPS: <https://github.com/wrf-model/WPS>
- NOAA ARL utilhysplit: <https://github.com/noaa-oar-arl/utilhysplit>

Recommended setup

Use Linux or WSL2 for the scientific stack.

```
mkdir -p third_party
```

```
git clone --recurse-submodules https://github.com/wrf-model/WRF.git third_party/WRF
```

```
git clone https://github.com/wrf-model/WPS.git third_party/WPS
```

```
git clone https://github.com/noaa-oar-arl/utilhysplit.git third_party/utilhysplit
```

Do not require WRF compilation for the first dashboard demo.

Demo-first strategy

Start with:

Bundled demo data

↓

FastAPI

↓

Next.js dashboard

↓

72h forecast

↓

Source + inversion

↓

What-if slider

Then replace individual modules with research-grade implementations.

Important scientific positioning

WRF-Chem is the physics/chemistry engine used for offline/batch experiments and generation/validation of training data.

The interactive dashboard uses a fast surrogate model.

XGBoost performs local bias correction against observations.

The demo must never imply that WRF-Chem is being rerun for every slider movement.